

QUDA multigrid 算法实现

从近零模聚合到递归粗格求解的源码证据链

源码分析汇报

QUDA 参考源码快照

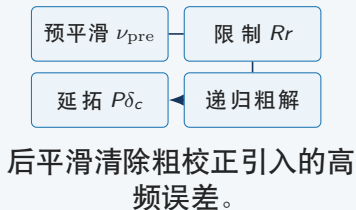
2026 年 8 月 27 日

结论先行: QUDA 的 MG 是一个可递归、可验证的自适应预条件器

已确证的主链



运行期闭环



当前判断与边界

确证: 源码内建四类算子一致性检查。

推断: 递归 MG 负责低频, 平滑器负责局部/高频。

未验证: 本次未运行 GPU/MPI, 暂无吞吐或收敛曲线。

汇报重点

不是只列出 “V-cycle” 名称, 而是追踪 P/R 、Galerkin 粗化、预条件粗算子、求解器工厂与验证钩子如何在源码中闭合。

来源: lib/multigrid.cpp:1131-1224, 745-1128; 图为源码控制流抽象。

问题与验收标准：粗空间必须保留难解的低频分量

物理/数值对象

格点 Dirac 算子在小质量或强耦合背景下会出现难以由局部平滑器消除的长程误差。MG 的实现目标是用近零模张成的粗空间表示这部分误差，再把局部高频误差交给平滑器。

关键映射必须满足维数一致性：

$$V: \mathcal{V}_c \rightarrow \mathcal{V}_f, \quad R: \mathcal{V}_f \rightarrow \mathcal{V}_c, \\ D_c = RDP.$$

来源：参数定义：include/quda.h:638-835；粗场维数：lib/dirac_coarse.cpp:121-169；验收实现：lib/multigrid.cpp:785-880, 883-1068。

输入/控制量

影响实现的参数

层级
聚合

n_level, 最多由编译常量限制
geo_block_size、
spin_block_size

粗空间
迭代

n_vec、precision_null
nu_pre、nu_post、
cycle_type

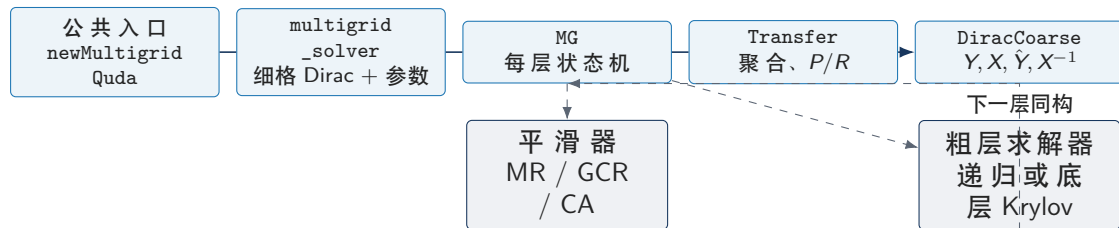
正确性

$P^\dagger P$ 、 $D_c - RDP$ 、normality

成功判据

P 与 R 在聚合子空间上近似互伴；粗算子与隐式投影一致；预条件变体与 $X^{-1}D$ 一致；求解器残差达到设定容差。

总体架构：公共 API 只负责装配，MG 负责递归控制



对象与存储

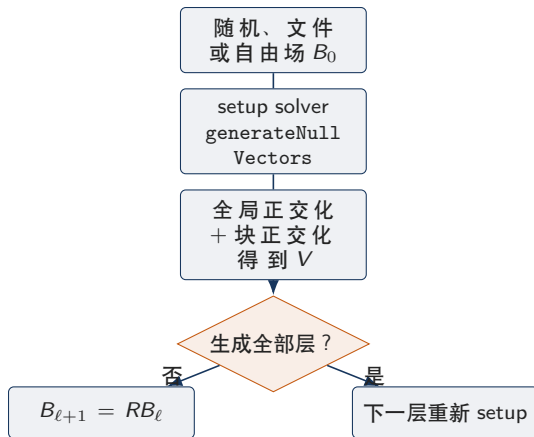
每个非底层保存近零模 B 、Transfer、粗链接 Y 、局部项 X ；需要偶奇预条件时再构造 $\hat{Y}$ 与 X^{-1} 。

物理维数检查

粗链接的内部维数是 $N_s^c \times N_c^c$ ，其中 $N_c^c = N_{\text{vec}}$ ， $N_s^c = N_s / \text{spin_block}$ ；因此聚合不仅缩小时空网格，也重新组织自旋/色自由度。

来源：入口装配：lib/interface_quda.cpp:2853-2927；对象构造：include/multigrid.h:75-217；粗场维数：lib/dirac_coarse.cpp:121-145。

设置阶段：近零模由平滑/特征路径产生，并可递归传递



来源：入口分支：lib/multigrid.cpp:36-62；setup 参数与 solver 选择：lib/multigrid.cpp:1275-1363；刷新/向下传递：lib/multigrid.cpp:1365-1451。

四类入口

代码按优先级支持：读取、特征、迭代生成近零模、自由场；setup 可选 CG/CA-CG、GCR、MG。

关键实现判断

use_init_guess=YES 让生成/刷新从现有 B 开始；MG setup 还把自身作为 GCR 预条件器，形成自举闭环。

聚合与块正交化：把局部近零模变成可用的粗基

几何映射

逐维整除得到 $x_c^\mu = \lfloor x_f^\mu / b_\mu \rfloor$ 。先建 `fine_to_coarse`，再按 (x_c, x_f) 排序为连续访问的 `coarse_to_fine`。块尺寸须整除局部格点，当前索引避免奇数粗维。

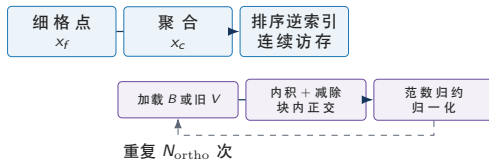
块内迭代细化

经典 Gram-Schmidt 重复 N_{ortho} 次：

$$v_j \leftarrow v_j - \sum_{i < j} \langle v_i, v_j \rangle v_i, \quad v_j \leftarrow v_j / \|v_j\|.$$

一个线程块处理一个聚合；`mVec` 个向量以 `tile` 批量归约。

来源：映射：lib/transfer.cpp:200-241；块正交化：include/kernels/block_orthogonalize.cuh:141-253；参数语义：include/quda.h:647-657。



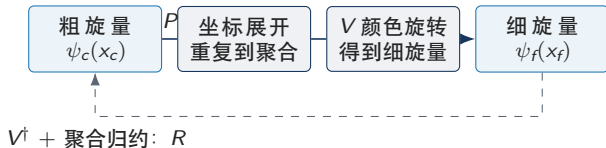
精度工程

`block_ortho_two_pass` 在低精度下先估计尺度，再定标计算；这改善定点/半精度下的正交化稳定性，但不是自适应误差证明。

延拓/限制是同一局部基的两种方向: P 展开, R 投影

$$(P\psi_c)(x_f) = V(x_f) \psi_c(x_c),$$

$$(R\psi_f)(x_c) = \sum_{x_f \in x_c} V^\dagger(x_f) \psi_f(x_f).$$



局部正交性的意义

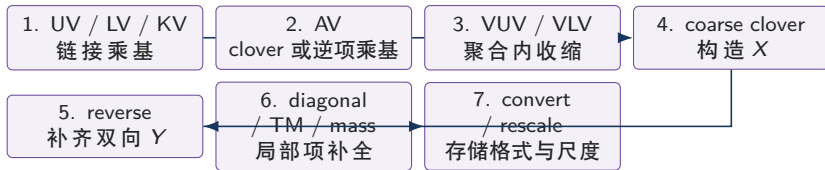
若每个聚合内 $V^\dagger V \approx I$, 则 $R \approx P^\dagger$, 并且粗变量的量纲/自旋色分块与构造时一致。

GPU 实现

Prolongator 先由几何映射取粗点, 再以 V 混合颜色; Restrictor 对 V 取共轭并用 block reduction 聚合。MMA 变体只替换矩阵乘实现, 不改变数学接口。

来源: 延拓: `include/kernels/prolongator.cuh:65-132`; 限制: `include/kernels/restrictor.cuh:90-124`; MMA 接口: `include/multigrid.h:650-687`。

Galerkin 粗化被拆成可调的 GPU 计算图，而不是一次黑盒矩阵乘



$$D_c = RDP \implies Y_\mu \sim V^\dagger U_\mu V, \quad X \sim V^\dagger C V + I + (\text{质量/扭质量项}).$$

Wilson 自旋结构

DeGrand-Rossi 基下 $P_\mu = (1 \pm \gamma_\mu)$ 产生自旋对角/非对角块；非对角项显式调用 `gamma.apply`，换方向翻转符号。

两级复用

`coarsecoarse_op.hpp` 通过 `from_coarse` 复用 `calculateY`；`fine/coarse spin-color` 差别由模板参数吸收。

来源: `lib/coarse_op.cuh:1043-1457`; `include/kernels/coarse_op_kernel.cuh:1067-1138`;
`lib/coarsecoarse_op.hpp:5-32`。

偶奇预条件把粗算子变成 $\hat{Y}, X^{-1}$ 的 Schur 补系统

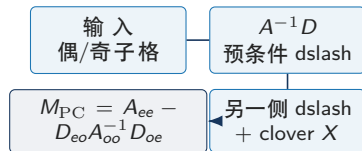
粗 dslash 的局部形式

$$D_{\text{hop}}\psi(x) = \sum_{\mu} [Y_{-\mu}(x)\psi(x+\mu) + Y_{\mu}^{\dagger}(x-\mu)\psi(x-\mu)].$$

GPU 内核分开前/后向 gather: 内部点直取, 边界点取 halo; $X\psi(x)$ 同 kernel 加入局部项。

$$\begin{aligned}\hat{Y}_{+\mu} &= X^{-1}Y_{+\mu}, \\ \hat{Y}_{-\mu} &= Y_{-\mu}X^{-\dagger}.\end{aligned}$$

来源: 粗 dslash/PC: include/kernels/dslash_coarse.cuh:126-289、lib/dirac_coarse.cpp:530-649; 预条件链接: include/kernels/coarse_op_preconditioned.cuh:60-120。



实现上的关键区别

DiracCoarsePC::Dslash 使用 $\hat{Y}, X^{-1}$; createCoarseOp 明确: PC 系统向下粗化 $\hat{Y}$, 不是原始 Y 。

平滑器消除局部误差，外层 GCR 允许每次使用不同的 MG 预条件效果



MR 平滑器

每步用 $\alpha = (Ar, r) / (Ar, Ar)$ 更新 $x \leftarrow x + \omega \alpha r$,
 $r \leftarrow r - \omega \alpha Ar$; 源码用一次复合归约同时产生所需内积/范数。Schwarz 时按红黑或加性块处理。

来源: MG 控制流: lib/multigrid.cpp:1171-1221; 平滑器装配: lib/multigrid.cpp:273-337; MR: lib/inv_mr_quda.cpp:65-190; GCR: lib/inv_gcr_quda.cpp:307-407。

MG 的 pre/post 装配

createSmoother() 将 nu_pre 与 nu_post 分别写入两个 Solver; 预平滑返回残差, 后平滑以当前解为初值。

GCR 外层

每轮先调用预条件器 $K(r)$, 再用 Ap_k 建立正交方向; 达到 Krylov 长度或可靠更新点时重算真残差并重启。

精度与硬件路径是同一算法的实现层，而非另一套 MG 数学

低精度零空间

$$B, V$$

precision_null

低精度存储减小带宽
与显存压力；块正交
化支持 two-pass。

Halo 与通信

$$V \xrightarrow{\text{exchangeGhost}} V_{\text{halo}}$$

粗 dslash 对边界点走
ghost；平滑器可单独
设置 halo 精度和
global reduction。

MMA / Tensor Core 定点尺度

setup_use_mma

dslash_use_mma

transfer_use_mma

替换 tile 矩阵乘与布
局，接口保持不变。

$$M \rightarrow sM$$

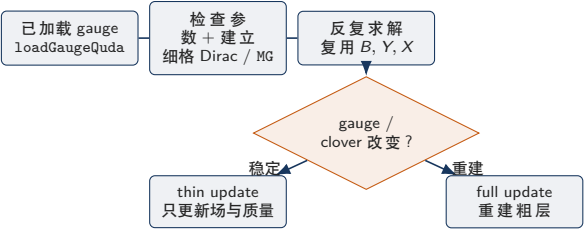
先找最大幅值，再
convert/rescale；链接
和局部项分别处理尺
度。

工程判断

代码通过模板参数、字段布局、halo 交换和尺度核，把同一 (RDP) 计算图映射到 CPU、CUDA、低精度和 MMA 路径；但“更快”需要硬件上的基线实测，不能由源码静态阅读推出。

来源：two-pass 参数：include/quda.h:650-669；ghost 与 setup：lib/coarse_op.cuh:1051-1062；尺度/转换：lib/coarse_op.cuh:1227-1368, 1443-1457；MMA 参数：include/quda.h:659-669。

公共 API：一次 setup，随后可 thin/full update 并复用 MG 状态



API	作用
<code>newMultigridQuda</code>	参数检查、细层与递归构造
<code>updateMultigridQuda</code>	thin 或 full 更新
<code>dumpMultigridQuda</code>	导出零空间向量
<code>destroyMultigridQuda</code>	释放递归资源

限制条件

当前入口要求外层使用 direct solve；平滑器 solve type 只接受 direct / direct-PC。粗层可选择递归 MG 或显式 Krylov 包装器。

来源：构造与限制：lib/interface_quda.cpp:2853-2927；更新分支：lib/interface_quda.cpp:2946-3055；API 声明：include/quda.h:1256-1289。

验证与风险：源码有强校验钩子，但本次没有动态通过记录

检查对象	代码判据	目的
零空间投影	$(1 - PP^\dagger)v_k$	检查细向量是否落在聚合子空间
粗空间闭合	$(1 - P^\dagger P)\eta_c$	检查 (R/P) 是否保持粗变量
Galerkin	$D_c - P^\dagger DP$	native 粗算子与显式投影一致
PC dslash	$\hat{D}_c - A^{-1}D_c$	预条件链接是否正确
normality	$\eta^\dagger M^\dagger M \eta$ 虚部相对量	检查求解器所需算子性质

本次状态

确证：上述检查均有源码实现，且按精度设置容差。

未验证：本次未启动 CUDA/MPI，不能声称检查通过。

ASQTAD 长链接或聚合尺寸过小时，代码会跳过部分 verify。



来源：verify: lib/multigrid.cpp:745-832, 866-999, 1001-1068; ASQTAD: lib/multigrid.cpp:896-925。

阶段结论与下一步：先把实现正确性变成可重复的运行证据

已经回答

确证：从 API 到 `MG::operator()` 的依赖链闭合。

确证：(P/R)、Galerkin、PC Schur、MR/GCR 均有直接源码锚点。

推断：低频由粗空间承担，高频由平滑器承担。

尚未回答

未获得：不同 n_{vec} 、聚合尺寸、cycle type 的收敛曲线。

未获得：CPU/GPU、MMA/非 MMA、单 rank/MPI 的时间和带宽。
这些不能从静态代码行号推导。

建议验收产物

1. 单 GPU Wilson/Clover smoke: verify 全部通过。
2. 'multigrid_evolve_test': 重复 update 后记录真残差。
3. 'multigrid_benchmark_test': 固定格点与 RHS, 给出无 MG 基线。

静态证据
本报告

单卡/单 rank
正确性

MPI + 多配置
收敛

性能与参数
决策

来源：测试入口：tests/multigrid_evolve_test.cpp:103-179、tests/multigrid_benchmark_test.cpp:135-153；
以上为建议执行顺序，非本次实测结果。

附录 A: 15 步分析框架覆盖 (1/2)

步骤	必答问题	本报告归宿	证据状态
1	目的与准备	为什么需要粗空间; 准备 gauge、参数与层级	本页主题定义
2	输入是什么	n_{level} 、聚合、零空间、求解器参数	include/quda.h:638-835
3	输出是什么	V 、 Y/X 、 $\hat{Y}/X^{-1}$ 、递归解	lib/dirac_coarse.cpp:121-223
4	整体框架	API $\rightarrow$ MG $\rightarrow$ Transfer/Coarse/solver	lib/interface_quda.cpp:2853-2927
5	关键细节	映射、CGS、 P/R 、Galerkin、Schur	lib/transfer.cpp:200-241 等
6	如何正确执行	new $\rightarrow$ setup $\rightarrow$ solve $\rightarrow$ update/destroy	include/quda.h:1256-1289
7	实际执行情况	本次只读源码与现有测试入口, 未启 CUDA/MPI	未验证: 无运行日志
8	实际输出情况	本次交付为源码分析 slides; 无性能/残差数据	确证: PDF/TeX 产物

范围说明

主题聚焦 QUDA multigrid 算法实现; 数值结果页不伪造曲线, 动态执行缺口在主体“验证与风险”页显式保留。

附录 A：15 步分析框架覆盖（2/2）

步骤	必答问题	本报告归宿	证据状态
9	结果分析	从 verify 钩子解释 P/R 、Galerkin、PC 一致性	lib/multigrid.cpp:785-1068
10	综合分析	低频/高频分工与模板化 GPU 计算图的关系	推断：由控制流与内核结构推出
11	结尾总结	MG 是可递归、可验证的自适应预条件器	确证/推断
12	结尾评价	优点：层级复用；缺点：配置与动态验证成本高	源码结构 + 未验证项
13	补充说明	ASQTAD/低精度/硬件路径的边界	lib/multigrid.cpp:896-925 等
14	参考源	逐页来源脚注与本页代码索引	本报告全部来源
15	附录与附件	直接引用源码片段；不附造假运行数据	确证：见后两页

置信度分层

“源码做了什么”是确证；“为什么有利于低频误差”是算法层推断；“在特定 GPU 上提升多少”必须等待实测。

附录 B：递归求解主循环（源码直引）

```
1 // restrict to the coarse grid
2 transfer->R(r_coarse, residual);
3
4 // recurse to the next lower level
5 (*coarse_solver)(x_coarse, r_coarse);
6
7 // prolongate back to this grid
8 if (!presmoothen) {
9     transfer->P(inner_solution_type == outer_solution_type ? x : x(parity), x_coarse);
10 } else { // we must sum to the presmoothen solution
11     auto res = inner_solution_type == outer_solution_type ? cvector_ref<ColorSpinorField>(r) :
12                                                            cvector_ref<ColorSpinorField>(r)(parity);
13     transfer->P(res, x_coarse);
14     xpy(res, inner_solution_type == outer_solution_type ? x : x(parity));
```

阅读提示

上段展示 $R \rightarrow$ 粗解 $\rightarrow P$ 的顺序，以及有预平滑时将粗校正加回当前解的分支；prepare/reconstruct 负责偶奇/全场约定。

来源：原文件：lib/multigrid.cpp:1192-1205；代码未转写。

附录 B: setup 与 Galerkin 计算图 (源码直引)

```
1 // launch solver for each source
2 if (B.size() % param.n_vec_batch != 0) errorQuda("Bad
3 batch size %d", param.n_vec_batch);
4 for (auto i = 0u; i < B.size(); i += param.n_vec_batch) {
5     if (param.mg_global.setup_type
6         == QUDA_TEST_VECTOR_SETUP) { // DDalphaAMG test
7         vector idea solving against the vector
8         copy({b.begin(), b.begin() + param.n_vec_batch}, {B.
9 begin() + i, B.begin() + i + param.n_vec_batch});
10         zero(x); // with zero initial guess
11     } else {
12         copy({x.begin(), x.begin() + param.n_vec_batch}, {B.
13 begin() + i, B.begin() + i + param.n_vec_batch});
14         zero(b);
15     }
16 }
```

来源: lib/multigrid.cpp:1383-1393

```
1 // First compute the coarse forward links if needed
2 if (bidirectional_links) {
3     for (int d = 0; d < nDim; d++) {
4         logQuda(QUDA_VERBOSE, "Computing forward %d UV and VUV\
5 n", d);
6         y.setDimension(d);
7         y.setDirection(QUDA_FORWARDS);
8         y.setComputeType(COMPUTE_UV); // compute U*V product
9
10        if (uv.Precision() == QUDA_HALF_PRECISION) {
11            y.setComputeMax(true);
12            y.apply(device::get_default_stream());
13            y.setComputeMax(false);
14            auto uv_max = *arg.max_h;
15            uv.Scale(uv_max);
16        }
17    }
18 }
```

来源: lib/coarse_op.cuh:1166-1179

对应关系

左: 按 batch 将 B 送入 setup solver; 右: 按方向先做 UV, 再做 VUV, 累加粗链接。

附录 B: Galerkin 与 PC 正确性检查 (源码直引)

```
1   logQuda(QUDA_SUMMARIZE, "Checking  $0 = (D_c - P^\dagger D P)$  (native coarse operator to emulated operator)\n");
2   zero(tmp_coarse);
3   zero(r_coarse);
4
5   #if 0 // debugging trick: point source matrix elements
6       tmp_coarse.Source(QUDA_POINT_SOURCE, 0, 0, 0);
7   #else
8       spinorNoise(tmp_coarse, *rng, QUDA_NOISE_UNIFORM);
9   #endif
10
11   // put a non-trivial vector on the fine level as well
12   transfer->P(tmp1, tmp_coarse);
13
14   // the three-hop terms in ASQTAD can break the verification depending on how we're coarsening the operator
15   // and if the aggregate size is too small in a direction
16   bool can_verify = true;
```

证据边界

该片段确证 QUDA 将 native 粗算子与显式 $P^\dagger D P$ 对比；误差数值和通过状态仍需真实运行日志。

来源：原文件：lib/multigrid.cpp:883-899；代码未转写；动态结果本次未验证。

附录 C：源码证据索引

主题	直接证据
层级对象与参数	include/multigrid.h:75-217; include/quda.h:638-835
API 装配/更新	lib/interface_quda.cpp:2853-3055
零空间生成	lib/multigrid.cpp:36-62, 1275-1472
聚合映射/正交化 (P/R)	lib/transfer.cpp:25-54, 200-241; include/kernels/block_orthogonalize.cuh:141-253
Galerkin 粗化	include/kernels/prolongator.cuh:65-132; include/kernels/restrictor.cuh:90-124
	lib/coarse_op.cuh:10-27, 1043-1463
	include/kernels/coarse_op_kernel.cuh:1067-1138, 1768-2031
粗 dslash / PC	include/kernels/dslash_coarse.cuh:126-375; lib/dirac_coarse.cpp:506-649
平滑器/外层求解器	lib/inv_mr_quda.cpp:65-190; lib/inv_gcr_quda.cpp:307-428
验证与测试入口	lib/multigrid.cpp:745-1128; tests/multigrid_evolve_test.cpp:103-179

报告结论只使用上述源码锚点；性能、收敛和硬件收益留待动态测试。